The National University of Lesotho

Department of Mathematics and Computer Science

Faculty of Science and Technology

CS4430: Distributed Database Systems

DISTRIBUTED DATABASE DESIGN

Coordinator: Mr. Khobatha Setetemela

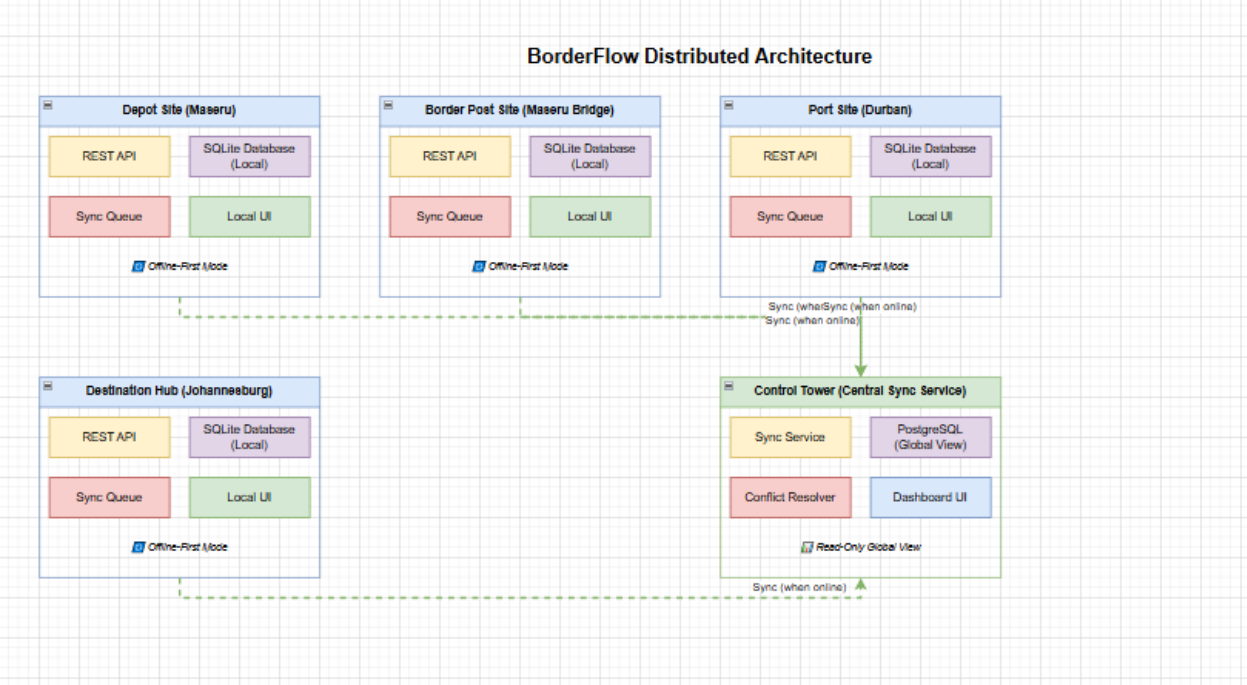
Due: April 2026

Team Members

Student Number	Name
200900706	Moleleki H.G.
202201934	Nkolokoane L.N.
202201176	Mohlapisi B.I.
202201173	Ramochele M.B.
202200990	Motaung M.
202201992	Makoetje K.G.M.
202201955	Leiee L.J.
202201074	Ramainoane M.A.R.

BorderFlow: Distributed Database Design

Architecture



2. Architecture Rationale

2.1 Design Decisions and Justification

Multi-Master Replication: All sites accept writes independently it prioritizes availability over strong consistency (AP from CAP theorem). Essential for offline-first operation where network partitions are expected.

Control Tower as Central Sync Hub: Simplifies discovery and reduces peer-to-peer complexity. Each site only needs to know one endpoint. Avoids gossip protocol overhead and split-brain scenarios.

SQLite per Site: Lightweight, zero-configuration, embedded database. Perfect for edge deployments. No separate database server process needed. Survives process restarts.

PostgreSQL at Control Tower: Robust, mature relational database for global view. Supports ACID transactions for conflict resolution. Handles concurrent sync requests reliably.

Push-Based Synchronization: Sites proactively push changes when connectivity is restored. Simpler than pull-based with polling. Guarantees that changes eventually reach the hub.

REST APIs: Platform-agnostic, simple to implement, easy to debug. Standard HTTP with JSON payloads. Works across firewalls and network boundaries.

3. Fragmentation Strategy

Fragmentation is the process of dividing the centralized database into smaller pieces (fragments) that are stored at different sites. BorderFlow uses a combination of horizontal fragmentation and derived horizontal fragmentation.

3.1 Fragmentation Type: Horizontal Fragmentation

Horizontal fragmentation divides a relation by rows (tuples). Each site stores only the rows that are relevant to its operations. The fragmentation predicate is based on the site_id attribute, which identifies which site originated each record.

Fragmentation Predicate:

```
-- Fragment definition for containers table:

containers_DEPOT   = σ(site_id = 'depot')   (containers)
containers_BORDER  = σ(site_id = 'border')  (containers)
containers_PORT    = σ(site_id = 'port')    (containers)
containers_HUB     = σ(site_id = 'hub')     (containers)

-- Same predicate applies to trips, milestones, incidents:
trips_DEPOT        = σ(site_id = 'depot')   (trips)
milestones_BORDER  = σ(site_id = 'border')  (milestones)
... etc.

-- Control Tower holds the UNION of all fragments (global view):
containers_GLOBAL  = containers_DEPOT
                   ∪ containers_BORDER
                   ∪ containers_PORT
                   ∪ containers_HUB
```

Completeness:

Every tuple in the global containers relation belongs to exactly one site fragment based on where it was created (site_id). The union of all fragments reconstructs the complete centralized relation — no tuple is lost.

Disjointness:

Since site_id is mutually exclusive across sites (a record created at depot has site_id='depot' and cannot simultaneously have site_id='border'), the fragments are disjoint. No tuple appears in more than one primary fragment.

Reconstruction:

```
-- Reconstructing the global containers relation:
containers = containers_DEPOT
           ∪ containers_BORDER
           ∪ containers_PORT
           ∪ containers_HUB

-- This is executed as a SQL UNION at the Control Tower:
SELECT * FROM containers;  -- PostgreSQL global view
```

3.2 Derived Horizontal Fragmentation

Trips, Milestones, and Incidents are derived fragments — they inherit their fragmentation from the parent Containers table via the container_id foreign key relationship.

```
-- Derived fragment: milestones at Border site

milestones_BORDER = milestones ⋈ containers_BORDER
```

```
                    (where milestones.site_id = 'border')


-- In practice, each site only stores milestones it recorded:
milestones_SITE_X = σ(site_id = 'X') (milestones)
```

Table	Fragment at Depot	Fragment at Border	Fragment at Port	Fragment at Hub	Global (Control Tower)
containers	site_id='depot'	site_id='border'	site_id='port'	site_id='hub'	UNION of all
trips	site_id='depot'	site_id='border'	site_id='port'	site_id='hub'	UNION of all
milestones	site_id='depot'	site_id='border'	site_id='port'	site_id='hub'	UNION of all
incidents	site_id='depot'	site_id='border'	site_id='port'	site_id='hub'	UNION of all
sync_queue	Local only	Local only	Local only	Local only	Not stored
processed_ops	Not stored	Not stored	Not stored	Not stored	Stored here only

4. Fragment Allocation

Allocation defines where each fragment is physically stored. BorderFlow uses a primary copy allocation strategy: each fragment has one primary copy at the originating site and one replicated copy at the Control Tower.

4.1 Allocation Table

Fragment	Primary Site	Technology	Replicated To	Access Pattern
containers_DEPOT	Origin Depot :8001	SQLite	Control Tower	Read/Write (offline-first)
containers_BORDER	Border Post :8002	SQLite	Control Tower	Read/Write (offline-first)
containers_PORT	Port Site :8003	SQLite	Control Tower	Read/Write (offline-first)
containers_HUB	Dest. Hub :8004	SQLite	Control Tower	Read/Write (offline-first)
containers_GLOBAL	Control Tower :8000	PostgreSQL	None	Read-only global view
sync_queue	Each site locally	SQLite	Not replicated	Write local, push to CT
processed_ops	Control Tower only	PostgreSQL	None	Idempotency check only

Allocation Rationale:

- **Edge sites:** SQLite at edge sites: zero-configuration, embedded, survives process restarts without a server process
- **Control Tower:** PostgreSQL at Control Tower: ACID transactions for reliable conflict resolution, handles concurrent sync requests

- **Locality:** Each site stores only its own fragment ;minimizes storage, maximizes locality of reference
- **Global view:** Control Tower stores the global union ;enables full visibility for management and compliance without querying all sites

5.1 Replication Model: Multi-Master with central Hub

BorderFlow implements multi-master replication — all four edge sites accept writes independently without coordination. This prioritizes availability over strong consistency, following the AP properties of the CAP theorem (Gilbert & Lynch, 2002).

Replication follows a push-based model: when a site detects restored connectivity, it pushes all pending operations from its local sync_queue to the Control Tower. The Control Tower acts as the single synchronization hub, sites do not communicate directly with each other.

Replication Property	BorderFlow Choice	Justification
Consistency model	Eventual consistency	Sites operate offline; strong consistency would block all writes during partition
Replication direction	Push-based (site → Control Tower)	Simpler than polling; sites know when they have pending data
Topology	Hub-and-spoke (all sites → Control Tower)	Avoids peer-to-peer complexity and split-brain scenarios
Write model	Multi-master (all sites accept writes)	Sites must function independently during network partitions
Sync frequency	Every 10 seconds when online	Balance between freshness and network overhead

5.2 Synchronization Protocol

Each site maintains a sync_queue table in its local SQLite database. Every write operation (INSERT, UPDATE) is assigned a unique operation_id (UUID) and appended to the queue. The sync worker runs as a background asyncio task:

```
SYNC PROTOCOL (per site, runs every 10 seconds):


1. IF site_online = FALSE:
      skip — queue accumulates locally
2. SELECT * FROM sync_queue WHERE synced = 0
      ORDER BY created_at LIMIT 50
3. IF no pending operations: CONTINUE
4. POST /sync/receive → Control Tower
```

```
      body: { site_id, operations: [batch] }

5. Control Tower processes each operation:

      a. CHECK processed_operations for operation_id

         → IF EXISTS: skip (idempotency)

      b. RESOLVE conflicts (Last Write Wins)

      c. APPLY to PostgreSQL global DB

      d. INSERT into processed_operations


6. Control Tower returns { accepted, skipped, conflicts }

7. Site marks operations as synced = 1

      UPDATE sync_queue SET synced = 1

      WHERE operation_id IN (batch)
```

6. Conflict Resolution Strategy

Conflicts arise when two sites update the same record independently while disconnected. BorderFlow implements Last Write Wins (LWW) using version vectors combined with timestamp tiebreaking.

6.1 Conflict Detection

When the Control Tower receives an incoming update for a record that already exists in the global database, it compares the version numbers:

```
FUNCTION resolve_conflict(table, record_id, incoming):

  existing = SELECT * FROM {table} WHERE id = record_id

  IF existing IS NULL:
      RETURN 'INSERT'   -- new record, no conflict

  IF incoming.version > existing.version:
      RETURN 'UPDATE'   -- incoming is newer, apply it

  IF incoming.version == existing.version:
      -- Concurrent update: same version, use timestamp
      IF incoming.updated_at > existing.updated_at:
          RETURN 'UPDATE'   -- incoming has later timestamp
      ELSE:
          RETURN 'SKIP'     -- existing is newer, discard incoming

  IF incoming.version < existing.version:
      RETURN 'SKIP'     -- incoming is stale, discard
```

6.2 Conflict Types and Resolutions

Conflict Type	Example Scenario	Resolution Strategy	Correctness Argument
Concurrent Updates	Depot and Border both update the same container status while offline	LWW: higher version wins; timestamp tiebreak for equal versions	Version counter increments on every write — concurrent writes at different sites have the

			same version but different timestamps
Duplicate Operations	Network retry causes same operation sent twice to Control Tower	Idempotency: operation_id checked against processed_operations; duplicate silently skipped	UUID operation_id is generated once at source — identical operation_id = identical operation
Missing Dependencies	Milestone arrives before its parent container is synced	INSERT OR IGNORE — milestone stored; container will arrive in next batch	SQLite foreign keys disabled at edge sites; Control Tower uses eventual reconciliation
Semantic Conflicts	Depot says DELIVERED, Border says IN_TRANSIT for same container	Last Write Wins by timestamp — most recent update wins; flagged in sync log	Operations log maintained for audit; manual resolution available via dashboard
Assignment Conflicts	Same driver assigned to two containers while sites are offline	Second assignment rejected at source — duplicate trip check on container_id	Each site checks for active trips before INSERT; prevents at source

7. Cross-Site Workflow with Correctness Argument

7.1 Complete Consignment Workflow

The following workflow demonstrates the complete lifecycle of a container across all four sites, including an offline scenario at the Border Post:

Step	Actor	Site	Action	Local DB State	Sync State
1	Depot Controller	Depot :8001	Register container BF-2026-001	containers: INSERT, version=1	Queued → synced to CT
2	Depot Controller	Depot :8001	Assign driver Thabo to trip → Border	trips: INSERT	Queued → synced to CT
3	Depot Clerk	Depot :8001	Gate-out: status → IN_TRANSIT	containers: UPDATE version=2	Queued → synced to CT
4	—	Border :8002	Network partition begins (OFFLINE)	sync_queue accumulates	Sync paused
5	Border Agent	Border :8002	Record CUSTOMS_CLEARED milestone	milestones: INSERT	Pending (offline)
6	Border Agent	Border :8002	Update status → CLEARED, version=3	containers: UPDATE	Pending (offline)
7	—	Border :8002	Network restored (ONLINE)	sync_queue: 2 pending ops	Sync resumes
8	Sync Worker	Border :8002	Push batch to Control Tower	sync_queue: synced=1	CT receives 2 ops

9	Port Agent	Port :8003	Record VESSEL_LOADED milestone	milestones: INSERT	Queued → synced to CT
10	Hub Clerk	Hub :8004	Confirm delivery → DELIVERED	containers: UPDATE version=4	Queued → synced to CT

References

Gilbert, S. and Lynch, N., 2002. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2), pp.51-59.

DeCandia, G. et al., 2007. Dynamo: Amazon's Highly Available Key-value Store. Proceedings of SIGOPS, pp.205-220.

Özsu, M.T. and Valduriez, P., 2020. Principles of Distributed Database Systems. 4th ed. Cham: Springer.

Madapparambath, G. and McKendrick, R., 2024. The Kubernetes Bible. 2nd ed. Birmingham: Packt Publishing.